EcoRoute Fleet: Product & Technical Build Specification

Version 1.0 · 31 Oct 2026 · Built from prototype v5, the Web Features doc, the Project Proposal, the Literature Review and the Verification Report

0. How to use this document

This is a single source of truth you can hand to an AI coding assistant (Claude Code, Cursor, v0, etc.) to build the real, deployable website. It captures everything your prototype does, calls out everything the prototype only fakes, and adds what is missing (backend, database, auth, simulator, real algorithms, deployment).

Tags used throughout

- **[PROTOTYPE]**: already visible in `ecoroute_prototype_v5.html`.
- **[ADDED]**: missing from the prototype and added by this spec.
- **[FUTURE]**: deliberately out of scope for the FYP (listed so nobody builds it by accident).

Recommended workflow

1. Give the AI this whole document once, then ask it to implement **one phase at a time** (Section 14). Do not ask for the entire site in one prompt.
2. Use the master prompt in Appendix C as the first message.
3. After each phase, run the acceptance checks in Section 13 before moving on.

Golden rules for the builder (also listed in Appendix C)

- Never hard-code demo numbers in UI code. Every number on screen comes from the database or from a calculation over database rows.
- Every emission, saving or credit figure is labelled **Estimated**. A figure is only called **Measured** if it was really measured by a benchmark run stored in the database.
- Follow the UI wording rules in Section 9.6 exactly.

1. Project summary

1.1 What EcoRoute is

EcoRoute Fleet is a carbon-footprint tracker for logistics trucks. A **Driver app** (installable web app, PWA) runs on the truck as the "edge node": it receives vehicle telemetry (speed, RPM, fuel rate, load, location), estimates fuel use and CO_2 on the device, detects idling and likely congestion, proposes a lower-emission route with a lightweight Genetic Algorithm (GA), keeps working offline, and uploads hash-chained records when the network returns. A **Manager website** shows fleet status, per-truck monthly and daily reports, eco-route savings, CO_2 allowances, alerts and data integrity.

The project title in the proposal is **Decentralized Edge-Based Carbon Footprint Tracker for Smart Logistics** (QUEST Nawabshah, BSIT, supervisor Dr. Baqar Ali Zardari).

1.2 The four research contributions the product must demonstrate

These come from the Literature Review and the Proposal. The website must make each one **visible and measurable**, because examiners will look for them.

#	Contribution	How the website shows it
C1	Lightweight GA running on the vehicle edge node (population about 20, early stopping, target 1.5 to 2 s), triggered by idling or congestion	Driver app eco-route card; GA timing stored per run; Research page
C2	Store-and-forward telemetry (local queue, outbox pattern, resend after reconnect) with full recovery after a simulated 10-minute outage	Offline badge, queue counter, Sync history, Research page outage test
C3	On-device emission model validated against the real Vehicle Energy Dataset (VED)	Research page validation tab (MAE, RMSE, MAPE, R^2)
C4	Controlled cloud vs edge comparison (latency, bandwidth, failure behaviour, route quality, resource use) plus SHA-256 hash-chain tamper evidence	Research page benchmark tab; Data Integrity page with a tamper demo

1.3 Scope

In scope (build this)	Out of scope [FUTURE]
Manager website (13 pages from the Web Features doc plus extras in the prototype)	Real vehicle hardware and live OBD-II dongles (we build a clean adapter interface and a simulator)
Driver PWA acting as the edge node	A native Android/iOS app (the proposal excludes it; the PWA covers the Android-phone edge test)
Telemetry simulator that generates realistic trips, congestion, outages and tampering	Government or carbon-registry integration; official credits
Real backend: auth, database, ingest API, rollups, integrity verification	Blockchain, ATECC608A secure chip, federated learning, TinyML forecasting (appear in the earlier Verification Report ideas but not in the final proposal)
Real GA, real emission model, real hash chain	Live traffic data feeds (traffic is simulated and must be labelled so)
Research page: edge vs cloud, outage recovery, hash-chain tamper test, VED validation results	Payments, billing, multi-company marketplace

2. How your materials fit together and what I changed

2.1 What each file contributed

File	Used for
`ecoroute_prototype_v5.html`	Visual design, navigation, screens, sample data, interaction behaviour, validation rules, i18n list
`EcoRoute_Fleet_Web_Features.docx`	Page list W1 to W13, feature wording, UI wording rules, build priority

File	Used for
`Student_Project_Proposal_Form_Final.docx`	Scope, tech stack intent, measurable outcomes, honesty rules ("targets are not results")
`Literature_Review__draft_for_your_proposal_.pdf`	The four research gaps and the algorithm parameters (population about 20, 1.5 to 2 s, 10-min outage, WAL + outbox + MQTT, SHA-256)
`researchpapers.pdf` and `Research_Papers_Verification_Report.pdf`	Related-work positioning (closest works: Odeh et al. 2025 roadside-edge GA; Chandra Priya et al. 2025 blockchain emission monitoring; Kulkarni et al. 2026 centralized web tracker)

2.2 What prototype v5 really is

Prototype v5 is a **front-end mock**. Be clear about this so the AI does not copy its shortcuts:

- All numbers come from a deterministic pseudo-random function and hard-coded arrays (8 trucks, 5 alerts, 5 hash rows). Nothing is computed from telemetry.
- Some numbers contradict the seed data. Example: the dashboard shows **Cargo moved 18.4 t**, but the seeded trucks sum to 17.1 t (active trucks) or 18.9 t (all trucks). The real build must **calculate** totals.
- "+3% vs yesterday", "1 h 35 m time saved", "94 credits forecast" and the early-warning popup (fires on a timer every 14 seconds) are hard-coded or fake.
- The hash chain table and the **Verify integrity** button are display-only. No SHA-256 is computed.
- The GA, early-warning risk score and VED validation do not exist yet. The "eco route" is two fixed numbers (18.4 km / 19 kg vs 19.1 km / 3 kg).
- State is kept in JavaScript variables and `localStorage` (fuel logs, trucks, allowances, offline maps) and an IndexedDB outbox. There is no server.
- Demo logins: manager `aamir.joyo@ecoroute.com` / `demo1234`; driver `PK-094` / PIN `1234`. These must only exist as **seed accounts in demo mode**.

2.3 Conflicts found and the decision taken

#	Conflict	Decision
1	Web Features doc: **1 credit = 1 tonne CO_2** (credits = kg / 1000). Prototype's per-truck allowance widget: **"1 credit = 1 kg CO_2"**	Store everything in **kg CO_2e**. Fleet-level "Estimated credits" = kg / 1000 (project convention). Per-truck allowance is a **kg limit** and the UI says "kg CO_2 allowance", not credits. Fleet monthly allowance = sum of truck allowances unless the manager sets an override.
2	Proposal says **no dedicated mobile app, no live vehicle sensors, no government integration**. Prototype has a Driver App, an OBD-II panel and Offline Maps	Driver app is a **PWA at `/driver`** (the "Android smartphone edge node" in the budget). OBD-II is a **pluggable telemetry source**: `simulator` (default) and `obd-bluetooth` [FUTURE]. The OBD badge works in both.
3	Proposal stack: Python, SQLite WAL, Mosquitto MQTT, Streamlit. You want **Vercel**	See the mapping table in 2.4. Python stays for offline **research scripts** (VED validation, benchmark analysis) using NumPy/Pandas.
4	Prototype derives diesel litres **from CO_2** (L = CO_2 / 2.68). Reality is the reverse	Real build: **fuel litres are the primary measurement** (integrated fuel rate), CO_2 = litres × emission factor.
5	Prototype uses a fixed demo date (12 Oct 2026)	Use the real current date, in the company time zone (`Asia/Karachi`).
6	Prototype Leaflet map uses public OpenStreetMap tiles	Fine for the FYP demo with attribution. For anything heavier, use a tile provider with a free tier (MapTiler, Stadia) via MapLibre.
7	Prototype translates only 31 UI strings into 7 languages	Build proper i18n for **English and Urdu (RTL)** first. Other languages are optional [FUTURE].
8	Prototype removed the **EV Stations** page	Keep it removed. **Vehicle Profiles** stays (Fuel, Hybrid, EV). EV tailpipe CO_2 = 0 with the note "grid emissions not included".

#	Conflict	Decision
9	Citation hygiene: Cheng and Jia (2024) is linked in `researchpapers.pdf` to a *Sustainability* 14(21) URL but referenced in the Literature Review as *Appl. Sci.* 14, 2024 (DOI 10.3390/app14219956). Bai et al. is 2025 in one file and 2024 in the other	Not a website issue, but check each DOI resolves to the paper and year you cite before submission.

2.4 Proposal stack mapped to a Vercel-friendly stack

Vercel runs short-lived serverless functions. It cannot host a long-running Mosquitto broker, a Python daemon, a writable SQLite file or a Streamlit server. The mapping below keeps the **same ideas** with different tools.

Proposal component	Vercel-friendly replacement	Why it still satisfies the proposal
Simulated telemetry stream (Python)	TypeScript **simulator module** that runs in the browser (driver PWA) and in a seed script	Same signals: speed, RPM, fuel rate, load, location
Local Python emission + idling logic	Same logic in a shared TypeScript package, run on the device	Processing happens at the edge node
Lightweight GA (Python)	TypeScript GA in a **Web Worker** on the device. A Python reference implementation lives in `/research` for cross-checking	Edge execution, population about 20, early stop
SQLite with WAL, local queue, transactional outbox	**IndexedDB outbox** (via Dexie): atomic write of record and outbox entry in one transaction, ordered by sequence number	Same pattern: durable local write first, send later

Proposal component	Vercel-friendly replacement	Why it still satisfies the proposal
MQTT (Mosquitto)	**HTTPS batch ingest** with idempotency keys and retry with backoff. Optional MQTT-over-WebSocket bridge to a hosted broker [FUTURE]	Same store-and-forward behaviour; works on Vercel
SHA-256 hash chain	Web Crypto `SubtleCrypto` on the device, Node `crypto` on the server for verification	Same algorithm
Streamlit dashboard	**Next.js** manager website	Same information, production-grade UI
NumPy / Pandas analysis, VED validation	Python scripts and notebooks in `/research`; results exported as JSON and imported into the `validation_runs` table for display	Keeps the stack stated in the proposal
Cloud comparison version	**Cloud mode**: telemetry goes to a Vercel function that computes risk and runs the same GA, then returns the decision	Gives a fair edge vs cloud experiment

3. Users, roles and permissions

Role	Who	Can do
`MANAGER`	Fleet manager of one company	Everything in the manager website for their company: fleet, reports, settings, add trucks and drivers, verify integrity
`DRIVER`	Truck driver	Driver PWA only: start and end trips, see own data, accept or decline eco-routes, log fuel, see own alerts

Role	Who	Can do
`EXAMINER` [ADDED]	Supervisor or examiner	Read-only access to the manager website and Research page; can trigger benchmark runs. Protects the demo data from accidental edits
`ADMIN` [ADDED]	Developer team	Create companies and manager accounts, seed or reset demo data. Not shown in the UI menu; protected route `/admin`

All data belongs to a `company_id`. Every query is filtered by the signed-in user's company. The default seeded company is **AJ Logistics**.

4. Manager website: pages and features

Priority tags follow the Web Features doc: **Must** (demo-critical), **Should**, **Nice**. All pages below exist in the prototype unless tagged [ADDED].

4.0 Global shell (every page after login)

- **Sidebar (228 px)** with groups. FLEET MANAGEMENT: Dashboard, Fleet & Trucks, Live Map, Multi-stop Planning. INSIGHTS: Route Savings, Reports, Carbon Credits, Fuel & Fleet Logs, Alerts, Data Integrity, Research [ADDED]. WORKSPACE: Vehicle Profiles, Drivers [ADDED], Settings. Active item = soft green pill. Brand block: logo mark, "EcoRoute Fleet", tagline "Drive smart. Cut carbon.". Bottom block: "Demo data · Not live telemetry" (only in demo mode), manager name, Log out.
- **Top bar (60 px)**: breadcrumb "EcoRoute / Page", global search [ADDED] (trucks, drivers, dates), **network badge** ("ONLINE MODE" or "OFFLINE MODE · n queued"), mode pill ("Demo mode" or "Live"), unread-alerts bell with count, avatar menu (Settings, Log out), theme toggle (light/dark), language menu.
- **Footer**: "© 2026 EcoRoute Fleet · AJ Logistics · Support · Privacy · Terms · Data last synced {relative time} · v{version}". Support, Privacy and Terms need real pages [ADDED].
- **Mobile (below 820 px)**: sidebar hidden; floating bottom nav with Home (Dashboard), Fleet, Map, Carbon and a **More** bottom sheet for the rest.
- **States on every table and chart**: loading skeleton, empty message ("No new alerts.", "No trips yet today."), error ("Something went wrong. Try again." with a retry button).

- **Demo Control Panel** [ADDED]: floating drawer visible to `EXAMINER` and `ADMIN` (and any user while `DEMO_MODE=true`). Buttons: simulate network loss, simulate OBD-II disconnect, inject congestion on the active truck, run a 10-minute outage test, tamper with a stored record, reset demo data, simulator speed ×1 / ×10. This panel is how you will show every research claim live.

4.1 W1 Manager Login (`/login`) · Must

- Split layout: left a scenic hero image (store it as a static file, **do not inline base64**) with eyebrow "EcoRoute Fleet Intelligence", headline "Every mile. Smarter. Cleaner.", text "Plan efficient routes, monitor fleet activity and understand estimated fuel and carbon savings, all in one place.", chips Eco-routing / Fleet insights / Offline-ready app, tagline "Drive smart. Cut carbon.".
- Right card: eyebrow "Welcome to EcoRoute", title "Welcome back", "Sign in to your fleet workspace." Fields: Email address, Password with Show/Hide eye [ADDED; doc requires it], Remember me, Forgot password?, **Sign in →**, divider "SECURE FLEET ACCESS", "Need access? Contact your administrator."
- Errors: "Please enter a valid email." and "Email or password is incorrect." (same message for wrong email or wrong password).
- Real behaviour [ADDED]: server-side auth, rate limit (5 failures per 15 min per email+IP), forgot-password email flow (demo mode shows "Reset link: simulated"). Never pre-fill credentials outside demo mode.

4.2 W2 Fleet Dashboard (`/dashboard`) · Must

Header: eyebrow "FLEET OVERVIEW · TODAY", title "Fleet Dashboard", subtitle "Estimated figures · Today, {date}", pill "● Live · updated {n} min ago", **Sync** button, **date picker** (default Today; changing it re-queries everything) [from doc].

Blocks, top to bottom:

1. **Quick actions**: Add truck, Multi-stop Planning, Fuel & Fleet Logs, Vehicle Profiles, Reports.
2. **Six KPI cards**: Estimated CO_2 today (kg, with "±x% vs yesterday" computed), Estimated credits used (CO_2 kg / 1,000; sub-text "1 credit = 1 tonne CO_2"), Diesel used (L), Active trucks ("6 / 8" with "1 idle · 1 offline"), Cargo moved (t, sub-text "Declared cargo"), Distance (km).

3. **Eco-route savings today (hero card)**: label "ECO-ROUTE SAVINGS TODAY (ESTIMATED)", title "Small route changes. Measurable impact.", three figures CO_2 saved / Diesel saved / Time saved, button **View details** → Route Savings.
4. **Monthly credit budget** card: "{used} / {allowance}" with progress bar; line "Early warnings today: N · accepted by drivers: M" [from doc, project's key feature].
5. **Fleet CO_2 Credit Allowance** widget: fleet total (Assigned / Used / Remaining kg) then one row per truck with usage bar, status pill (On track / Warning / Critical / Exceeded), text "{used} / {limit} kg CO_2 Used ({p}% Consumed - {d} Days Remaining) · {left} kg left" and an **Edit limit** link (modal, see Section 10).
6. **Last 24 hours** chart with CO_2 / Diesel toggle (hourly rollup; current hour amber) and **Last 7 days CO_2** chart with the daily target line.
7. **Trucks today** table (top 5 by CO_2; columns Truck, Status, Distance, CO_2, Diesel, Integrity; row click opens Truck Detail) and **Recent alerts** (latest 4, link "All alerts").
8. **Top routes by savings**: ranked horizontal bars, "Ranked by estimated CO_2 saved".

Refresh every 5 to 10 seconds only while the tab is visible; show "updated n min ago" from real timestamps.

4.3 W3 Fleet & Trucks (`/fleet`) · Must

- Summary chips: "8 trucks", "6 active", "1 idle", "1 offline" (computed).
- Controls: search (truck or driver), Status filter (All, Active, Idle, Offline), Integrity filter (Verified, Pending, Check needed) [from doc], date selector, **Add truck** button, **Export CSV** (exports the filtered rows).
- Table: Truck (id + driver name beneath), Plate, Status, Distance, CO_2, Diesel, Cargo, CO_2 saved, Last sync, Integrity chip. Pagination 25 per page. Row click and truck-name click open Truck Detail. Hint text: "Click a truck name to see its report."
- **Add truck modal**: License plate, Model, Fuel type (Diesel, CNG, Hybrid, EV), Driver (optional), Monthly CO_2 Credit Allowance in kg, Starting odometer (optional). All validation rules in Section 10.

4.4 W4 Truck Detail (`/fleet/[truckId]`) · Must

- Back link "← Fleet". Header card: "{TRUCK_ID} · {model}", subtitle "Plate {plate} · Driver {name} · Last sync {n} min ago", status chip, integrity chip, button **Open Live Map** (opens `/live?truck=id`).
- Tabs (URL query `?tab=`): **Overview, Monthly Report, Trips, Savings**. The truck stays the same when tabs change.

- **Overview**: four KPIs (Estimated CO_2 today, Estimated credits, Diesel today, Declared cargo); **Today by hour** chart with CO_2 / Diesel toggle; **Eco-route savings today** card (CO_2 saved, diesel saved, time saved, idle time); **Latest route** mini-map showing planned route, eco-route and the early-warning point.
- **Trips tab**: TRIP | FROM → TO | TIME | DISTANCE | CO_2 (trip id like `TRIP_KHI_HYD_102`).
- **Savings tab**: TIME | ORIGINAL | ECO ROUTE | SAVED | ACCEPTED, for this truck only.

4.5 W5 Monthly Report (tab, `/fleet/[truckId]?tab=monthly&month=YYYY-MM`) · Must

- Month picker. Summary cards: CO_2, Estimated credits, Diesel, Cargo moved, Days active, Eco-route savings. **Compare with last month** chip (±% vs previous month) [suggested in doc].
- **Daily CO_2** bar chart with the daily target line (setting "Daily CO_2 target per truck", default 450 kg); today's bar highlighted amber.
- **Day by day** table: DAY, TRIPS, DISTANCE, CO_2, CREDITS, DIESEL, CARGO, CO_2 SAVED, INTEGRITY, "View day" link. Today's row has a **Live** badge. Hint: "Click any day to see its full details."
- **Export PDF** and **Export CSV** for this truck and month.

4.6 W6 Day Detail (`/fleet/[truckId]/day/[YYYY-MM-DD]`) · Must

- Back link "← {TRUCK_ID} · Monthly Report". Title "{d} {Month} {year}" (with "· Live" for today). ‹ **Previous / Next** › switchers keep the same truck.
- Six KPIs: Estimated CO_2, Credits, Diesel, Declared cargo, Distance, Idle time.
- **24-hour chart**: hourly CO_2 bars plus diesel line; idle hours shaded light amber [from doc].
- Tables: **Trips**; **Eco-route events** (Time, Original, Eco, Saved, Accepted Yes/No); **Early warnings of the day** (level Low / Medium / High, place, time, driver's decision, risk score) [from doc]; **Alerts of the day**.
- **Data integrity** card: status chip, "{n} records · last check {time}", note "Integrity checks detect changes after recording. They do not confirm that the original sensor readings were accurate."

4.7 W7 Reports (`/reports`) · Should

- Controls: Report type (Emissions summary, Fuel, Savings, Cargo), Period (Weekly / Monthly), Month, Trucks (multi-select). Summary strip: fleet totals for CO_2, credits, diesel, cargo, savings.
- **CO_2 by truck per week** stacked chart; truck table (Truck, Driver, CO_2 kg, Credits, Diesel L, CO_2 saved, Integrity; truck name opens detail).
- Buttons: **Export PDF**, **Export CSV**, and a "Reports and logbooks" card with Carbon savings report (CSV), Trip logbook (CSV), Carbon report (PDF).
- **Send me this report every month** is shown as a disabled chip labelled "coming soon" (optional later: Vercel cron + email).

4.8 W8 Live Map (`/live`) · Should

- Map with truck markers coloured by status: Active `#1E9E5A`, Idle `#F5A524`, Offline `#9AA7A2`. Planned route solid blue `#2B7BD6`, eco-route dashed green `#1E9E5A`, early-warning pins amber with a warning icon.
- **Layer toggles**: Routes, Early warning points, Congestion. Right panel: truck list with search, **selected truck card** (speed, today's CO_2, current route, last alert), button **Open truck report**.
- Label required when data is simulated: "OpenStreetMap tiles. Truck positions are simulated." Never write "live GPS" or "live traffic" for simulated data. Poll every 5 seconds.

4.9 Multi-stop Planning (`/planning`) · Should

- Draggable list of stops (start flag, numbered middle stops, end pin), per-leg distance, remove button (minimum 2 stops, message "Keep at least 2 stops"), add-stop selector (city list: Karachi, Hyderabad, Nawabshah, Sukkur, Multan, Lahore, Islamabad, Quetta; later free text with geocoding).
- **Optimise order** button runs the GA (Section 6.5). Toast "Stops reordered (estimated lowest-emission order)". Prototype only did nearest-neighbour by straight-line distance × 1.25; the real build uses the road graph and the fuel model.
- **Route summary** card (estimated): total km, selected vehicle profile, litres and kg CO_2 (EV shows "Tailpipe CO_2 0 kg (grid emissions not included)"), "Optimised order could save about {x} km / {y} kg CO_2".
- [ADDED] Save plan, assign it to a truck and driver; the driver sees it as the next trip.

4.10 W9 Route Savings (`/savings`) · Should

- Hero card "{MONTH} TO DATE (ESTIMATED)": CO_2 saved, Diesel saved, Time saved, Reroute events.
- **Original route vs Eco route** grouped bars (same emission model applied to both) [from doc]; **Cumulative CO_2 saved** line over the month.
- **Reroute events** table: DATE, TRUCK, TRIGGER (Early warning or Idle), CO_2 SAVED, DIESEL, TIME, ACCEPTED. Filters by truck and date. Footnote always visible: "All savings are estimates from the simulated on-device model."

4.11 W10 Carbon Credits (`/credits`) · Should

- Amber-bordered banner on top: **"Estimated usage only. Not an official carbon registry balance."**
- Cards: Used today, Used this month, Monthly allowance; progress bar; **month-end forecast** (average per day so far × days in month) worded "On track to use about N by month end (estimate)".
- **Credits used per day** chart with allowance line; **Credits by truck** horizontal bars; the Fleet CO_2 Credit Allowance widget (same as dashboard).
- **How is this calculated?** popover: "CO_2 = diesel litres × 2.68 kg/L. Credits = CO_2 kg / 1,000 (project convention: 1 credit = 1 tonne CO_2). Cargo weight and speed are not multiplied in."
- **Carbon Audit** table (Truck, Driver, CO_2 today, MTD CO_2, Allowance, Used %, Remaining, Integrity) with CSV and PDF export.

4.12 W11 Alerts (`/alerts`) · Should

- Tabs: All, Early warning, Idling, Offline, Integrity (+ "High CO_2 rate" from the doc). Filters: truck, date. Row: severity dot, type pill, title, truck · driver · time; dimmed once read.
- Buttons: **Mark as read**, **Mark all as read**, **Open truck report**. Row click opens a **detail drawer** with mini map, time, truck, recommended action.
- Example titles: "Early warning: slow traffic likely ahead", "Idling over 10 minutes", "Truck offline for 12 min", "Integrity check needed", "High estimated CO_2 rate", "CO_2 allowance Warning: 82% used". Alerts are created by the server from real events (Section 6).

4.13 W12 Data Integrity (`/integrity`) · Should

- Four cards: Trucks verified ("6 / 8"), Pending records, Check needed (with truck id), Last sync.
- Truck table: Truck, Last sync, Records received, Pending, Integrity chip, **First problem at record #** (only for Check needed). **View details** shows the hash chain: `#`, PREV HASH, HASH, STATUS, with the formula line "record_hash = SHA256(previous_hash + record data)".
- **Verify integrity** button runs the real verification (Section 6.7) and shows the result and timestamp. **Sync history** timeline: synced, partial sync, check failed.
- Info box, always visible: "Integrity checks detect changes after recording. They do not confirm that the original sensor readings were accurate."
- [ADDED] **Tamper demo** (examiner/admin only): pick a record, change one value in the database, press Verify and watch the chain break at that record.

4.14 Fuel & Fleet Logs (`/logs`) · Should

- Table: TRUCK, DATE / TIME, LITRES, ODOMETER, COST, STATUS (Synced or Pending sync). Buttons: **+ Fuel log** (modal), CSV, PDF. Driver-created offline entries stay "Pending sync" until uploaded.
- [ADDED] **Fuel reconciliation card**: for each truck compare litres refuelled between two logs with the estimated litres burned between them; show variance %, flag above a configurable threshold (default 15%). This is real-world evidence for your emission model.

4.15 Vehicle Profiles (`/vehicles`) · Nice

- Add/edit form: Name, Type (Fuel, Hybrid, EV), Efficiency (L/100 km for Fuel and Hybrid, kWh/100 km for EV). List with type icon, efficiency, "kg CO_2/km" (= efficiency / 100 × 2.68, EV shows "tailpipe CO_2 0 (grid not included)"), **Active** badge, buttons Set active / Edit / Delete (at least one profile must remain).
- Seed profiles: Hino 500 (Diesel) 30 L/100 km; Isuzu Hybrid 22 L/100 km; Eco EV Van 38 kWh/100 km.

4.16 W13 Settings (`/settings`) · Nice

- **Profile** (name, email, change password), **Company** (name, contact), **Targets & Alerts** (Monthly credit allowance e.g. 100; Idle alert after 10 min; Daily CO_2 target per truck 450 kg; Early warning sensitivity Low / Normal / High), **Trucks & Drivers** (assign driver

per truck), **Units & Language** (km, L, kg; language). **Save changes** shows toast "Changes saved". Settings feed the charts, alert thresholds and allowance bands.

4.17 Drivers (`/drivers`) [ADDED]

List, add, edit, deactivate drivers; assign truck; **Reset PIN** (shows the new PIN once). Needed because the prototype only shows a driver name string.

4.18 Research (`/research`) [ADDED] · Should (critical for the FYP defence)

Tabs: **Edge vs Cloud**, **Outage recovery**, **Hash-chain test**, **VED validation**, **GA performance**, **Related work**. Full spec in Section 6.8 to 6.10. Every number is either "Measured" (links to a stored run id and timestamp) or labelled "Simulated" or "Target". No number appears without one of these labels.

5. Driver app (PWA at `/driver`)

Installable (manifest, service worker), phone-first layout, works offline after first load. In the prototype it is shown inside a phone frame; in production it is a normal full-screen mobile web app (keep the phone-frame view only on the desktop demo page `/demo`).

5.1 Screens

Screen	Route	Contents
Login	`/driver/login`	Scenic header "Drive smart. Cut carbon.", Driver ID field, 4-digit PIN with on-screen keypad (1-9, C, ←, 0), dots indicator, "Driver ID or PIN is incorrect.", "Need help? Contact your fleet manager.", footer "Offline-ready · EcoRoute v1.0"

Screen	Route	Contents
Home	`/driver`	Greeting + truck chip ("TRUCK_PK_094 · TK-5021"); **CO_2 Allowance card** (assigned, consumed, remaining, % and band message); **OBD-II Live Telemetry** card; hero "TODAY SO FAR · ESTIMATED" kg CO_2 + credits; tiles Diesel, Distance, Declared cargo, Idle time; "Saved by eco-routes today"; Last 24 hours chart; quick actions (Fuel Log, Vehicle Profiles, Offline Maps, Trip); button **Start Trip** or **Continue Trip**
Trip setup	`/driver/trip`	Truck (read-only), From, To, **Declared cargo weight** stepper (−/+ 100 kg, big number), Eco-route suggestions toggle, readiness checklist (GPS on, battery %, location permission), **Start Trip**
Live trip	`/driver/live`	Speed hero (km/h) and "CO_2 now g/s"; tiles Trip CO_2, Trip diesel; **Traffic risk ahead** chip (Low/Medium/High); warning card "Slow traffic likely ahead in 900 m. Last exit in 1.2 km." with **See eco-route**; turn-by-turn card; mini-map; Voice alerts toggle; OBD-II card; **End Trip** (only when the truck is stopped: toast "Stop the truck to end the trip")
Eco-route decision	`/driver/eco`	Card "Early warning: slow traffic likely ahead – Decide before the exit in 1.2 km"; side-by-side **Current route** (km, min, diesel L, CO_2 kg) and **Eco route** with "Recommended" pill; savings hero "You save: 6 L diesel · 16 kg CO_2 · 11 min (Estimated)"; buttons **Take eco-route** and **Stay on current route**; text "This is a suggestion. You decide." Both choices are recorded as a route event with `accepted` true/false
Trip summary	`/driver/summary`	Total CO_2, diesel, distance; whether an eco-route was taken and its saving; "Records verified" chip; **Save and close**, **View day details**

Screen	Route	Contents
Day detail	`/driver/day`	Date, Live chip, totals, tiles, By-hour chart (CO_2/Diesel toggle), trips, eco-route savings, integrity chip
History	`/driver/history`	Month header with totals; day cards (CO_2, litres, saved) with cloud (saved) or hourglass (pending) icon
Impact	`/driver/impact`	"SAVED THIS MONTH (ESTIMATED)" hero, This week bars, Recent eco-routes, **Eco streak** (days in a row with an accepted eco-route or CO_2 under target)
Alerts	`/driver/alerts`	Own alerts only, offline notice "You are offline. Your data is saved on this phone."
Sync	`/driver/sync`	Network badge, tiles Queued records / Fuel logs / GPS points / Route updates, **Sync now**, "Simulate network loss (demo)" toggle
Fuel Log	`/driver/fuel`	Form (Date & time, Fuel litres, Odometer, Cost optional) and last 5 refills with Synced / Pending sync
Vehicle Profiles	`/driver/vehicles`	Same data as the manager page
Offline Maps	`/driver/offline-maps`	Map packs (Sindh corridor 182 MB, Punjab corridor 240 MB, Karachi city 64 MB) with Download / Remove; stored size. Real build caches map tiles or the local road graph in the service worker or IndexedDB; the prototype only simulates it
Profile	`/driver/profile`	Name, Driver ID, truck, Language, Units, Night mode, Voice alerts, simulate OBD-II disconnect (demo), Vehicle Profiles, Offline Maps, "Help: contact manager" (call link), version, **Log out**

Bottom tab bar: Home, Trip, History, Impact, Profile, with an animated selection indicator. Header: "EcoRoute Driver", network badge, alerts bell with count. A thin "Truck ECU link" bar under the header shows OBD-II Connected or Disconnected.

5.2 OBD-II Live Telemetry card

Tiles: Vehicle Speed (km/h), Engine RPM ("Standby" when stopped), Instant Fuel (L/100 km), Coolant Temp (°C), Engine Load ("{n}% Load (Idling / No Idling)"). When the OBD link is lost: all values show "--" and the message "No ECU data. Check that the OBD-II dongle is plugged in." and the toast "OBD-II signal lost. Check the dongle or OBD port. Telemetry is paused." Recording pauses and a gap marker is written to the hash chain so that the gap is explainable, not silent.

5.3 Behaviours to keep from the prototype

- Voice alerts through the browser speech API for early warnings and turn instructions; toggle on Live trip and Profile.
- Toast when the CO_2 allowance crosses a band: "CO_2 allowance Warning: 82% used" (also Critical, Exceeded).
- Offline: fuel logs, route events and GPS points are written to the outbox; the badge changes to "OFFLINE MODE · n queued"; when the network returns the badge shows "syncing…" and then a success toast "Network connection restored. Offlined trip logs and CO_2 telemetry data have been successfully synchronized with the Fleet Manager server."
- Night mode and language are stored per driver.

6. Core engines (the real logic behind the screens)

These engines are what the prototype only pretends to do. Put them in a shared TypeScript package (`packages/core`) so the **same code** runs on the device (Web Worker), in the cloud-mode API route and in the seed script. That sharing is what makes the edge vs cloud comparison fair.

6.1 Telemetry record

The device samples at 1 Hz in memory and **persists one record every 10 seconds** (configurable) to keep storage small. Fields:

Field	Type	Notes
`truckId` , `tripId`	string	

Field	Type	Notes
`seq`	integer	per truck, starts at 1, +1 per record, never reused
`ts`	ISO 8601 UTC	end of the interval
`lat`, `lon`	number (6 dp)	
`speedKmh`	number (1 dp)	interval average
`rpm`	integer	
`fuelRateLph`	number (2 dp)	interval average
`engineLoadPct`, `coolantC`	integer	
`cargoKg`	integer	**declared** by the driver, not measured
`idle`	0 or 1	engine on and speed below 5 km/h for the interval
`fuelMl`, `co2G`	integer	burned in this interval
`src`	`sim` or `obd`	where the numbers came from
`gap`	0 or 1	marks "telemetry paused" (OBD loss, app killed)
`prevHash`, `hash`	hex string	Section 6.7

6.2 Emission model (on-device)

- **Fuel first.** `fuelMl = fuelRateLph × interval_s / 3.6`. If the source provides engine fuel rate (OBD-II PID 0x5E) use it. Otherwise estimate it from speed, RPM, load and payload with the model below.
- **Parametric estimate when no fuel-rate sensor exists:** `L/h = p0 + p1·v + p2·v² + p3·(rpm/1000) + p4·load%`, then multiplied by a payload factor `1 + k·(cargoKg/1000)`. Coefficients `p0..p4` and `k` are stored per vehicle class in the

`vehicle_profiles` table, initialised from the rated L/100 km and refined by calibration against VED (light-duty) and against the fuel logs from real refills. State this limitation in the UI of the Research page.

- **Idle burn:** when `idle = 1` use the profile's idle rate (L/h, default 2.0 for medium trucks, editable).
- **CO_2:** `co2Kg = litres × EF`. Emission factors live in config with a source note: Diesel **2.68 kg/L** (project convention in your docs); Petrol (needed for VED validation), CNG per kg, Hybrid (uses the fuel actually burned) and EV (tailpipe 0, "grid emissions not included") are configurable. **Confirm every factor against your chosen published source before reporting it.**
- **Credits:** `credits = co2Kg / 1000`.
- **Do not double count:** cargo and speed influence fuel, so never multiply them into CO_2 again.
- The prototype's demo formula (`L/h = 8 + 0.38·speed`) is a placeholder and must not ship.

6.3 Idle detection

`idle = engineOn && speed < 5 km/h`. When idle lasts at least **Idle alert after** minutes (setting, default 10) create one **Idling** alert per episode; re-arm after the truck moves above 5 km/h for 60 s. Daily "Idle time" = sum of idle intervals.

6.4 Early-warning risk score (congestion ahead)

Evaluated every second on the device from rolling windows:

- `ahead` = `1 − expectedSpeedNext2km / freeFlowSpeed` from the congestion layer of the road graph (simulated, so it must be labelled "simulated traffic").
- `drop` = relative fall of current speed versus the 60-second average before it.
- `stopGo` = number of dips below 10 km/h in the last 120 s, divided by 4, capped at 1.
- `prior` = historical congestion prior for this segment and hour (0 to 1).

`score = clamp(0.45·ahead + 0.30·drop + 0.15·stopGo + 0.10·prior, 0, 1)`. These weights are an **initial heuristic** to be tuned on simulation.

Bands (from the Web Features doc): below 0.4 = Low, 0.4 to below 0.6 = Medium, 0.6 or higher = High. **Sensitivity** setting shifts the cut-offs: Low 0.5/0.7, Normal 0.4/0.6, High 0.3/0.5.

Trigger an early warning when the band reaches High (or Medium when sensitivity is High), a usable exit or detour exists within the look-ahead distance, and no warning was shown in the last 5 minutes. The warning text is "Slow traffic likely ahead in {m} m. Last exit in {km} km." It starts the GA (6.5) and, if a saving is found, shows the eco-route card.

Evaluation: the simulator knows where congestion zones really start. A warning is a true positive if a congestion zone starts within 2 km after the warning. Report precision, recall and mean lead distance on the Research page only after they are measured over a stored set of runs.

6.5 Genetic Algorithm eco-routing (contribution C1)

Road graph. `nodes` (junctions, exits, cities), `edges` (`lengthKm` , `freeFlowKph` , `roadClass` , optional gradient, `congestionFactor` per hour 0 to 1). Start with a hand-built **Sindh to Punjab corridor** graph: the eight cities in the prototype (Karachi 24.86/67.01, Hyderabad 25.39/68.37, Nawabshah 26.25/68.41, Sukkur 27.70/68.86, Multan 30.16/71.52, Lahore 31.55/74.34, Islamabad 33.68/73.05, Quetta 30.18/66.98) plus real junctions, bypasses and exits extracted once from OpenStreetMap. Ship it as static JSON in `/public/graph/` and cache it in the service worker so it works offline. Do not call external routing APIs from the GA.

Edge cost (shared with the emission model). For edge *e* with congestion *c*: `v = max(5, freeFlow·(1 − c))` , `time = length / v` , `fuel = L/h(v, cargo) × time × (1 + 0.6·c)` (stop-go penalty), CO_2 `= fuel × EF` . This is why a slightly longer eco-route can burn much less fuel than a shorter congested one (your 18.4 km vs 19.1 km example).

Problem A: reroute between the current position and the next stop.

- Chromosome: an ordered list of 0 to 3 **via-nodes** from the nearby sub-graph. Decode by joining time-weighted shortest paths origin → via_1 → via_2 → destination (with loop removal).
- Fitness (minimise): CO_2`_kg + λ_t·time_h + penalty` , where `λ_t` is configurable (default 0.5 kg per hour) and the penalty applies if the detour exceeds a configurable limit (default +25% distance) or the path is invalid.
- Parameters: **population 20**, max 40 generations, tournament size 3, crossover 0.8 (one-point on via lists), mutation 0.2 (replace, insert or delete a via-node), elitism 2.
- **Seeded initial population** (the benefit of good initial solutions shown by Cheng and Jia): shortest-distance path, fastest path under current congestion, the current route, and random fill.

- **Early stopping:** stop after 8 generations without improvement, hard time cap **2.0 s** (target 1.5 to 2 s). Return the best-so-far if the cap hits.
- **Suggest only if worth it:** show the eco-route only when it saves at least 3% CO_2 and at least 1 kg versus the current route.
- Runs in a **Web Worker** so the UI never freezes.

Problem B: multi-stop order (Multi-stop Planning page). Chromosome = permutation of the middle stops (start and end fixed), order crossover, swap/inversion mutation, fitness = sum of leg CO_2 from the graph. Compare against nearest-neighbour and, for 8 stops or fewer, brute force.

Every GA run is stored (`ga_runs`): trigger (early_warning, idle, manual, planning), mode (edge or cloud), population, generations, `elapsedMs` , baseline CO_2, best CO_2, saved kg / L / min, random seed, device label. This feeds Route Savings and the Research page.

Baselines for quality comparison: current route, Dijkstra shortest distance, Dijkstra fastest time under congestion.

6.6 Offline store-and-forward (contribution C2)

Device storage (IndexedDB via Dexie): `records` (primary key `truckId+seq`), `outbox` (`id` uuid, `kind` telemetry / fuel / route_event / gps / alert_ack, `payload` , `createdAt` , `attempts` , `nextAttemptAt` , `status` pending / inflight / acked / dead), `meta` (`lastSeq` , `lastHash` , `deviceId`).

- **Transactional outbox:** the record and its outbox entry are written in **one IndexedDB transaction**, before any network call.
- **Sender loop:** runs on the `online` event, when the tab becomes visible, every 15 s, and via Background Sync where supported. Do not trust `navigator.onLine` alone: check with a lightweight `GET /api/v1/health` (2 s timeout).
- **Batching:** up to 200 items or 256 KB, ordered by `seq` . `POST /api/v1/ingest/batch` with header `Idempotency-Key: <batchId>` .
- **Server reply** per item: `stored` , `duplicate` or `conflict` . On 2xx mark acked and delete. On 5xx or timeout, exponential backoff 1, 2, 4 … up to 60 s with jitter. On 4xx validation errors mark the item `dead` , keep it for inspection and raise an alert. On 401 refresh the token and retry.
- **Network simulation:** the Demo Control Panel flips a `NET.sim` flag that makes the sender behave as offline and the badge show "OFFLINE MODE · n queued".

- **Storage guard:** warn the driver when the queue exceeds 50,000 items or storage use exceeds 80% of the quota.
- **Success metric:** `recoveryRate = recordsOnServer / recordsGenerated` for the outage window; target 100% with 0 duplicates, measured with the **10-minute simulated outage** from the proposal.

6.7 SHA-256 hash chain (tamper evidence)

- Genesis `prevHash` = 64 zeros. `hash = hex(SHA256(prevHash + "\n" + JSON.stringify([truckId, tripId, seq, ts, lat, lon, speedKmh, rpm, fuelRateLph, engineLoadPct, coolantC, cargoKg, idle, fuelMl, co2G, src, gap])))`. Use an **array** (fixed order) and fixed decimal formatting so device and server always compute identical bytes. One shared function in `packages/core`.
- Device computes with Web Crypto (`crypto.subtle.digest`). Server recomputes on ingest and during verification with Node `crypto`.
- **Ingest check:** the first record of a batch must have `prevHash` equal to the stored last hash for that truck (or genesis). A mismatch or a `seq` collision with a different hash is stored and flagged `conflict`.
- **Verification job** (button, after each ingest, and daily cron): walk records by `seq` from the last checkpoint, check that `seq` is contiguous, the recomputed hash equals the stored hash and `prevHash` matches the previous record. Result states:
  - **Verified**: all good through the latest record.
  - **Pending**: missing `seq` numbers that are still inside the grace window (default 24 h) or the device still reports queued items.
  - **Check needed**: a hash mismatch, a duplicate `seq` with a different hash, or a gap older than the grace window. Store `firstBadSeq`.
- Save every run in `integrity_checks`, update the truck's integrity status, and create an "Integrity check needed" alert when it fails.
- **What it proves:** changes, deletions or re-ordering of records after they were chained are detectable. **What it does not prove:** that sensor readings were accurate, or that someone controlling the device did not rewrite the whole chain before the first sync. Optional hardening [FUTURE]: the server stores the tail hash of every synced batch, and a daily Merkle root is emailed to the manager.
- Wording: use "tamper-evident", never "tamper-proof" or "fraud-proof".

6.8 Telemetry simulator

A seeded generator (mulberry32 PRNG) used by the driver app "Simulation mode" and by the seed script:

- Follows a route over the graph, with target speed = free-flow × (1 − congestion), acceleration limit ±1.5 m/s², noise, depot dwell times, toll and junction stops.
- Signal formulas from the prototype are acceptable as a base: `rpm = 780 + 19·speed + noise`; `engineLoad = idle ? 8 : min(95, 15 + 0.9·speed)`; coolant moves slowly toward 78 to 92 °C.
- **Scenario flags** for reproducible demos: `congestionZone`, `outage(10 min)`, `obdLoss`, `tamper`. Scenario files are JSON so the same scenario can run in edge and cloud mode.
- **Seed script** creates 30 days of history for the 8 trucks in Appendix A by running the **real** pipeline (record, hash, ingest, rollup), so every chart and hash on screen is genuine. Deliberately create: TRUCK_PK_108 with a broken chain at one record (Check needed), TRUCK_PK_052 offline with pending records, TRUCK_PK_011 idle.

6.9 Edge vs Cloud experiment (contribution C4)

Same seeded scenarios, two modes:

- **Edge mode**: risk score and GA run on the device; only batched records and route events are uploaded.
- **Cloud mode**: the device streams samples to `POST /api/v1/cloud/decide`; the Vercel function runs the same risk and GA code and returns the decision.

Metric	How it is measured
Decision latency	sample time to decision visible (edge: compute time; cloud: round trip + server time; flag cold starts)
Bandwidth	total bytes up and down (from `PerformanceResourceTiming` and payload sizes)
Failure behaviour	number and share of decisions still made during the simulated outage
Route quality	CO_2 of the chosen route versus the three baselines
Resource use	main-thread long tasks, worker run time, memory where the browser exposes it

Metric	How it is measured
Recovery	records recovered after outage (6.6)

Run at least 30 repetitions per mode and network profile; report mean, median, p95 and a 95% confidence interval. Store raw runs in `benchmark_runs`. Show each result with its device, network profile, region and date. The proposal says the performance advantages are **hypotheses**, so word the page that way until the data is in.

6.10 VED validation (contribution C3)

Done **offline** in `/research` with Python (NumPy, Pandas), then imported:

- Dataset: the Vehicle Energy Dataset (University of Michigan; Oh, LeBlanc and Peng; real driving of several hundred vehicles around Ann Arbor over about a year). Download it yourselves from its official repository and keep it out of git. Check the column names and licence in the dataset's own documentation. It contains per-sample speed, RPM, MAF, absolute load, fuel rate for some vehicles, location and timestamps, plus a static table with vehicle type and engine data.
- Steps: select conventional (ICE) vehicles with a usable reference signal, resample to 1 Hz, run **the same TypeScript model** through a small Node CLI (or port the coefficients to Python and prove equality on a test file), integrate to litres per trip, compare with the reference litres.
- Metrics: MAE, RMSE, MAPE, R^2, total fuel error %, per speed band and per vehicle class, plus a Bland-Altman style bias plot. Convert to CO_2 with the petrol factor for validation and with the diesel factor for trucks.
- **Honest limits:** VED is mostly light-duty passenger cars in Michigan. It shows that the estimation method works on real telemetry; it does not prove truck coefficients. Pair it with the fuel-log reconciliation card (4.14).
- Output: a JSON file imported with `pnpm research:import validation.json` into `validation_runs` and shown on the Research page (run id, subset size, metrics table, estimated vs reference scatter, residual histogram).

Research page tabs summary: *Edge vs Cloud* (6.9), *Outage recovery* (6.6 test with a record counter before, during and after), *Hash-chain test* (6.7 tamper demo), *VED validation* (6.10), *GA performance* (distribution of `elapsedMs`, share of runs within 2 s, quality versus baselines, population-size ablation 10 / 20 / 40), *Related work* (table of the ten papers from the Literature Review with "what they did, limitation, how EcoRoute differs").

7. Architecture on Vercel

7.1 Stack decisions

Layer	Choice	Why
Framework	**Next.js** (App Router, TypeScript strict, latest stable; pin the exact version at scaffold time)	One repo for pages and API; first-class on Vercel
UI	Tailwind CSS + shadcn/ui (Radix) with the design tokens in Section 9; Recharts for charts; TanStack Table; React Hook Form + Zod	The prototype's hand-made CSS bars become real, accessible charts
Maps	`react-leaflet` with OpenStreetMap tiles (as in the prototype), or MapLibre with a free-tier tile provider	Keep attribution; do not hammer public OSM tiles
Database	**Postgres via the Vercel Marketplace (Neon)** with Drizzle ORM and the serverless driver	Vercel functions have no persistent writable disk, so SQLite files cannot be the production store
Auth	Auth.js (credentials) for managers and examiners; custom **driver PIN login** that issues a device token	Real sessions, hashed secrets
Password hashing	bcrypt (cost 12) or argon2id	Never store plain text, even for demo accounts
PWA and offline	Serwist (service worker), Dexie (IndexedDB), Web Workers for the GA	Edge-node behaviour in the browser
Background jobs	**Vercel Cron** calling `/api/cron/*` protected by `CRON_SECRET`	Daily verification, rollup repair, retention
Rate limiting	Upstash Redis via the Marketplace (or a Postgres counter table)	Login and ingest protection
Email	Resend (password reset, later scheduled reports)	Simple API

Layer	Choice	Why
i18n	`next-intl` with English and Urdu (RTL)	Prototype already has Urdu RTL
PDF/CSV	jsPDF + autoTable on the client (as in the prototype), CSV built server-side with a BOM	Avoids headless-Chrome PDFs on Vercel
Tests	Vitest, Playwright, fast-check	Section 13
Research scripts	Python (NumPy, Pandas) in `/research`	Matches the proposal; not deployed

7.2 Vercel realities the design already respects

- **No long-lived connections.** There is no MQTT broker or WebSocket server on Vercel. Live pages **poll every 5 to 10 seconds** only while the tab is visible. A hosted realtime service can replace polling later [FUTURE].
- **Cron limits.** On the Hobby plan Vercel cron jobs can run at most **once per day** (checked in Vercel's docs, June 2026), so the app never depends on frequent cron. Truck status, alerts, hourly and daily rollups are updated **at ingest time**; the daily cron only verifies, repairs and cleans up. Pro plans allow per-minute schedules.
- **Hobby is for non-commercial use** under Vercel's fair-use rules, which is fine for the FYP demo. A real company deployment needs a paid plan.
- **Function size and time.** Keep ingest batches at 256 KB or less (Vercel limits request bodies on functions) and set `maxDuration` explicitly: ingest 10 s, cloud-mode decision 10 s, report export 30 s.
- **Cold starts** affect cloud-mode latency. Record a `coldStart` flag in every cloud benchmark call and report it instead of hiding it.
- **Region.** Put the functions and the database in the same region, as close to Pakistan as available (for example Mumbai or Dubai; check current availability in Vercel docs) and note the region next to every benchmark result.
- **Read-only filesystem.** Never write files at runtime. Static assets (graph JSON, hero images, map packs) live in `/public` or Vercel Blob.

7.3 Repository layout

```
ecoroute/
  src/
    app/
      (public)/login, privacy, terms, support
      (manager)/dashboard, fleet/[truckId]/(day/[date]), live, planning,
                savings, reports, credits, logs, alerts, integrity,
                vehicles, drivers, settings, research
      driver/ (login, home, trip, live, eco, summary, day, history, impact,
               alerts, sync, fuel, vehicles, offline-maps, profile)
      demo/             phone-frame demo launcher (desktop only)
      admin/            seed/reset, validation import
      api/v1/...        route handlers (Section 8.2)
      api/cron/...      verify-integrity, rollup-repair, retention
    core/               pure TS shared by device, server, seed (no DOM, no DB)
      emission.ts  idle.ts  risk.ts  ga/  graph.ts  hashchain.ts
simulator.ts
    db/                 drizzle schema, migrations, queries, seed
    device/             dexie schema, outbox sender, worker wrappers, network
monitor
    workers/ga.worker.ts
    components/         ui, charts, tables, map, chips, modals
    i18n/               en.json, ur.json
  public/ graph/ icons/ hero/
  research/             python scripts, notebooks, README (VED steps)
  tests/  e2e/
  vercel.json  drizzle.config.ts  .env.example  README.md
```

7.4 Environment variables

Name	Purpose
`DATABASE_URL`	Pooled Postgres connection string
`AUTH_SECRET`	Session signing key
`DEVICE_TOKEN_SECRET`	Signs driver device tokens
`CRON_SECRET`	Vercel sends it as `Authorization: Bearer ...` to cron routes; reject any call without it

Name	Purpose
`DEMO_MODE`	`true` shows the demo control panel, seed accounts and "Demo mode" pill
`NEXT_PUBLIC_APP_NAME`, `NEXT_PUBLIC_APP_VERSION`	Footer text
`RESEND_API_KEY`, `MAIL_FROM`	Email
`UPSTASH_REDIS_REST_URL`, `UPSTASH_REDIS_REST_TOKEN`	Rate limiting
`NEXT_PUBLIC_TILE_URL`, `NEXT_PUBLIC_TILE_ATTRIBUTION`	Map tiles

7.5 Performance and storage

- Dashboards and reports read **rollup tables** (`hourly_summary`, `daily_summary`), never scan raw telemetry.
- Raw `telemetry_records` volume at one record per 10 s: about 2,880 per truck per day, about 23,000 per day for 8 trucks, roughly 7 MB per day. Keep **14 days of raw data** by default (retention cron), keep rollups and `integrity_checks` forever. Seeded history can use a coarser interval.
- Index `(truck_id, ts)` on telemetry; `(truck_id, date)` on daily summary; `(company_id, ts desc)` on alerts.

8. Data model and API

8.1 Tables

All tables carry `created_at`; tenant tables carry `company_id`. Names are suggestions; keep the columns.

Table	Key columns
`companies`	id, name, contact_email, timezone (`Asia/Karachi`), units, settings (monthly_allowance_override_kg, idle_alert_min = 10, daily_target_kg = 450, sensitivity = normal, fuel_variance_threshold_pct = 15)
`users`	id, company_id, role (MANAGER, DRIVER, EXAMINER, ADMIN), name, email (unique), password_hash, driver_code (e.g. `PK-094`), pin_hash, pin_failed_count, locked_until, truck_id, language, theme, is_active, last_login_at
`device_tokens`	id, user_id, truck_id, device_id, token_hash, expires_at, revoked_at
`vehicle_profiles`	id, company_id, name, type (fuel, hybrid, electric), efficiency_value, efficiency_unit, fuel_type, ef_kg_per_unit, idle_lph, model_coeffs (json), is_active
`trucks`	id (text, e.g. `TRUCK_PK_094`), company_id, plate (unique per company), model, fuel_type, vehicle_profile_id, driver_id, status (active, idle, offline), integrity_status (verified, pending, check_needed), odometer_km, monthly_allowance_kg, last_seen_at, last_sync_at, last_lat, last_lon, last_speed_kmh
`trips`	id (e.g. `TRIP_KHI_HYD_102`), truck_id, driver_id, origin, destination, declared_cargo_kg, started_at, ended_at, distance_km, fuel_l, co2_kg, idle_s, plan (json), status
`telemetry_records`	PK (truck_id, seq); trip_id, ts, lat, lon, speed_kmh, rpm, fuel_rate_lph, load_pct, coolant_c, cargo_kg, idle, fuel_ml, co2_g, src, gap, prev_hash, hash, received_at, batch_id
`ingest_batches`	id (= idempotency key), truck_id, device_id, item_count, stored, duplicates, conflicts, bytes, received_at, response (json)

Table	Key columns
`hourly_summary`	truck_id, hour_start, co2_kg, diesel_l, distance_km, idle_s, records
`daily_summary`	truck_id, date, trips, distance_km, co2_kg, diesel_l, cargo_kg, idle_s, co2_saved_kg, diesel_saved_l, time_saved_min, records, integrity_status
`route_events`	id, truck_id, trip_id, ts, trigger (early_warning, idle, manual, planning), location_name, lat, lon, original (km, min, L, kg), eco (km, min, L, kg), saved_kg, saved_l, saved_min, accepted, ga_run_id
`early_warnings`	id, truck_id, trip_id, ts, risk_score, level, place, distance_ahead_m, last_exit_km, decision (accepted, declined, ignored), simulated
`ga_runs`	id, truck_id, trigger, mode (edge, cloud), population, generations, elapsed_ms, baseline_co2_kg, best_co2_kg, seed, device_label
`alerts`	id, company_id, truck_id, type (early_warning, idling, offline, integrity, high_co2_rate, allowance), severity, title, body, recommended_action, ts, meta
`alert_reads`	alert_id, user_id, read_at
`fuel_logs`	id, client_id (unique, for offline idempotency), truck_id, driver_id, logged_at, litres, odometer_km, cost, status (synced, pending)
`integrity_checks`	id, truck_id, run_at, status, records_checked, first_bad_seq, detail (json), triggered_by
`sync_events`	id, truck_id, ts, kind (synced, partial, check_failed), records, pending
`trip_plans`	id, company_id, stops (json), vehicle_profile_id, total_km, co2_kg, optimised, assigned_truck_id
`benchmark_runs`	id, run_group, ts, scenario, mode (edge, cloud), network_profile, device_label, region, cold_start, metrics (json)
`validation_runs`	id, ts, dataset, subset (json), metrics (json), notes

Table	Key columns
`audit_log`	id, company_id, user_id, action, entity, entity_id, before, after, ip, ts

Derived truck status. `offline` when no heartbeat or record was received for more than 10 minutes (the server cannot tell "no signal" from "truck off"; the app keeps recording either way, which is the whole point). `idle` when the latest records show speed below 5 km/h for 2 minutes or more with the engine on. Otherwise `active`.

Rollups. After each accepted batch, upsert `hourly_summary` and `daily_summary` using only the records that were newly stored (duplicates are skipped, so nothing is double counted; late records land in their own hour). The daily cron recomputes today and yesterday from raw data to repair drift.

8.2 REST API (`/api/v1`, JSON, Zod-validated)

Conventions: auth cookie for managers, `Authorization: Bearer <deviceToken>` for devices; tenant comes from the session, never from the request; errors are `{ "error": { "code": "VALIDATION", "message": "...", "fields": {...} } }`; pagination `?page=1&pageSize=25`; times are ISO 8601 UTC, displayed in the company time zone.

Area	Endpoint	Notes
Auth	`POST /auth/login`, `POST /auth/logout`, `POST /auth/forgot`, `POST /auth/reset`	Rate limited
Driver auth	`POST /driver/login` (driverCode, pin, deviceId) → device token	5 failed PINs lock 15 min; bind token to truck and device
Health	`GET /health`	`{ ok, time }`, used by the sender's connectivity check
Ingest	`POST /ingest/batch`	Idempotent; see 8.3
Heartbeat	`POST /devices/heartbeat`	Position, queue size, OBD status
Fleet	`GET /fleet/summary?date=`, `GET /trucks?search&status&integrity&date&page`, `POST /trucks`, `PATCH /trucks/:id`	Summary powers the dashboard KPIs

Area	Endpoint	Notes
Truck	`GET /trucks/:id`, `GET /trucks/:id/overview?date=`, `GET /trucks/:id/monthly?month=`, `GET /trucks/:id/days/:date`, `GET /trucks/:id/trips`, `GET /trucks/:id/savings`	Day detail returns KPIs, hourly series, trips, route events, warnings, alerts, integrity
Charts	`GET /fleet/hourly?date=&metric=`, `GET /fleet/daily?days=7`	From rollups
Trips	`POST /trips` (start), `PATCH /trips/:id` (end), `GET /trips/:id`	End requires speed below 5 km/h
Route events	`POST /route-events`, `GET /route-events?truck&from&to`	Also accepted through ingest
Savings	`GET /savings/summary?month=`, `GET /savings/top-routes`	
Credits	`GET /credits/summary?month=`, `PUT /trucks/:id/allowance`	Bands in Section 10
Alerts	`GET /alerts?type&truck&date`, `POST /alerts/:id/read`, `POST /alerts/read-all`	
Integrity	`GET /integrity/summary`, `GET /integrity/trucks/:id/chain?from&to`, `POST /integrity/verify` (all or one truck), `POST /integrity/tamper` (examiner/admin, demo only)	
Fuel logs	`GET /fuel-logs`, `POST /fuel-logs` (idempotent by `clientId`)	Validation in Section 10
Vehicle profiles	`GET/POST/PATCH/DELETE /vehicle-profiles`	
Drivers	`GET/POST/PATCH /drivers`, `POST /drivers/:id/reset-pin`	
Planning	`GET/POST /trip-plans`, `POST /planning/optimise`	Optimise runs the GA on the server for the manager page

Area	Endpoint	Notes
Cloud mode	`POST /cloud/decide`	Benchmark only: risk + GA on the server, returns decision and timings
Reports	`GET /reports/data?type&period&month&trucks`, `GET /reports/export?kind=csv|pdf-data`	
Settings	`GET/PUT /settings`, `PUT /me`, `PUT /me/password`	
Research	`GET /research/benchmarks`, `POST /research/benchmarks/run` (examiner/admin), `GET /research/validation`, `POST /admin/validation/import`, `GET /research/ga-runs`	
Admin/demo	`POST /admin/seed`, `POST /admin/reset-demo`, `POST /demo/scenario`	Only when `DEMO_MODE=true` or role ADMIN
Cron	`GET /api/cron/verify-integrity`, `/rollup-repair`, `/retention`	`CRON_SECRET` required

8.3 Ingest contract (the most important endpoint)

Request (`Idempotency-Key: 6b1f…1`):

```
{
  "deviceId": "dev_a1",
  "truckId": "TRUCK_PK_094",
  "items": [
    { "kind": "telemetry", "record": { "seq": 1038, "tripId":
"TRIP_KHI_HYD_102", "ts": "2026-10-31T08:15:10.000Z",
      "lat": 25.396, "lon": 68.358, "speedKmh": 62.4, "rpm": 1960,
"fuelRateLph": 24.1, "engineLoadPct": 71,
      "coolantC": 86, "cargoKg": 3500, "idle": 0, "fuelMl": 66944, "co2G":
179410, "src": "sim", "gap": 0,
      "prevHash": "a4f5…", "hash": "e7b8…" } },
    { "kind": "route_event", "event": { "clientId": "re_91", "trigger":
"early_warning", "accepted": true } },
    { "kind": "fuel", "log": { "clientId": "f_77", "litres": 120.5,
"odometerKm": 83422 } }
  ]
}
```

Response: `{ "batchId": "...", "serverTime": "...", "results": [ { "ref": "telemetry:1038", "status": "stored" }, { "ref": "re_91", "status": "duplicate" } ] }`.

Server steps, in order: authenticate the device token and match the truck; validate with Zod (reject unknown fields); if the `Idempotency-Key` already exists return the saved response; recompute each hash and the `prevHash` linkage; insert with `ON CONFLICT (truck_id, seq) DO NOTHING` and compare hashes on conflict; update rollups, truck status and last sync; create alerts (idling, high CO_2 rate, allowance band, offline recovery); write `sync_events` (synced, or partial if some items failed); return per-item results. All in one transaction per batch.

9. Frontend design system and UI rules

The look is already decided by the prototype: calm green, soft cards, rounded pills, big readable numbers. Reproduce it with tokens, not copy-pasted CSS.

9.1 Design tokens

Token	Light	Dark
Primary green `--g`	`#0B6E4F`	same

Token	Light	Dark
Teal `--g2`	`#0F7F73`	same
Amber `--am`	`#F5A524`	same
Red `--rd` / Blue `--bl` / Violet (cargo)	`#D64545` / `#2B7BD6` / `#7A5AF8`	same
Background `--bg`	`#F4F7F6`	`#0E1A17`
Card `--cd`	`#FFFFFF`	`#16241F`
Border `--bd`	`#E1E8E5`	`#25382F`
Text `--tx` / Muted `--mu`	`#1B2B27` / `#5F6F6A`	`#E6F0EC` / `#9DB0A8`
Soft green `--soft`	`#EAF5F0`	`#1D3A2F`
Brand gradient	`linear-gradient(135deg, #0A5C44, #0F7F73)`	same
Card shadow	`0 8px 28px rgba(23,54,43,.07)`	subtle or none

- **Chips**: green (bg `#E1F5EA`, text `#1E7B4A`), amber (`#FFF1D6` / `#8A5A00`), red (`#FDE4E4` / `#B23232`), blue (`#E3EFFC` / `#1F5FA8`), grey (`#ECEFEE` / `#5F6F6A`).
- **Type**: Inter (fallback system-ui). Body 14 px / 1.45. Page title 26 px, weight 800, letter-spacing −0.6 px. Card title 16 px. Eyebrow 10 px uppercase, letter-spacing 1.4 px, green, weight 800. KPI value 28 px, weight 800. Hero number 44 px. Muted text 12 px.
- **Shape**: cards radius 14, inputs and buttons radius 10, pills 99, modals 14, KPI icon tile 34 × 34 radius 10 with a gradient, nav icon tile 30 × 30 radius 9.
- **Layout**: sidebar 228 px, top bar 60 px, content max-width 1300 px with 24 px padding (14 px on mobile), grid gap 14 px, KPI grid `repeat(auto-fit, minmax(180px, 1fr))`, two-column grid `minmax(320px, 1fr)`, mobile breakpoint 820 px, driver app content width 390 px.
- **Hero card**: brand gradient, white text, a faint circle outline at the top-right corner.
- **Dark mode** follows the system setting and can be overridden with the toggle (saved per user).

9.2 Components to build once and reuse

AppShell, Sidebar, TopBar, NetworkBadge (green "🟢 ONLINE MODE" / red "🔴 OFFLINE MODE · n queued" / "syncing…"), ObdBadge, KpiCard, HeroCard, StatusChip, DataTable (clickable rows, sorting, pagination, CSV), BarChart, GroupedBar, StackedBar, LineChart, ProgressBar with bands, Tabs, Toggle, Stepper, PinKeypad, Modal, Drawer, Toast, ConfirmDialog, EmptyState, ErrorState, Skeleton, MapView, DragList, LanguageMenu, ThemeToggle, DemoPanel, PopoverHelp ("How is this calculated?").

Nav icon tiles (gradient per item, keep or replace with Lucide icons in the same colours): Dashboard green, Fleet blue, Live Map teal, Planning violet, Route Savings pink, Reports blue, Carbon Credits green, Fuel Logs amber, Alerts red, Integrity purple, Vehicle Profiles orange, Settings grey.

9.3 Status colours

Value	Chip
Active, Verified, On track, Synced, Low	green
Idle, Pending, Pending sync, Warning, Medium	amber
Offline	grey
Check needed, Critical, Exceeded, High	red
Info (Recommended, vehicle chip)	blue

9.4 Charts and formatting

- Every bar or line chart has a title, unit, tooltip with the exact value, a target or allowance line where the data has one, and a text alternative (table toggle or `aria-label`).
- Numbers use `en-US` grouping (`1,234`); kg and L with 0 decimals, credits with 2, small litre values with 1. "CO_2" uses the subscript character. Durations read "1 h 35 m". Relative times ("2 min ago") come from real timestamps.
- Dates shown in the company time zone; stored in UTC.

9.5 Accessibility, i18n, responsiveness

- Keyboard reachable, visible focus, colour contrast AA in both themes, never colour alone (chips carry text), `prefers-reduced-motion` respected, live regions for toasts.
- English and Urdu from day one; Urdu sets `dir="rtl"` and mirrors the layout. All user-visible strings come from translation files. The 31 strings the prototype translates are the minimum for other languages.
- Test at 360, 390, 820 and 1280 px widths.

9.6 UI wording rules (from the Web Features doc, apply to app and web)

Labels: Estimated CO_2 · Estimated credits used · CO_2 saved · Diesel saved · Time saved · Diesel used (L) · Declared cargo (kg) · statuses Active / Idle / Offline · data checks Verified / Pending / Check needed · connection "Online" / "Offline · 12 pending" · early warning level Low / Medium / High.

Buttons: Start Trip · Continue Trip · End Trip · Take eco-route · Stay on current route · Sync now · View day · View details · Export PDF · Export CSV · Save changes · Mark as read · Mark all as read · Sign in · Log out.

Fixed messages:

When	Exact text
Offline	You are offline. Your data is saved on this phone and will upload automatically.
Back online	Back online. Syncing… then Sync complete.
Early warning	Early warning: slow traffic likely ahead. Last exit in 1.2 km.
Eco-route	This is a suggestion. You decide.
Credits	Estimated usage only. Not an official carbon registry balance.
Savings	All savings are estimates from the simulated on-device model.
Integrity	Integrity checks detect changes after recording. They do not confirm that the original sensor readings were accurate.

When	Exact text
Empty	No new alerts. / No trips yet today.
Error	Something went wrong. Try again.

Words that must never appear: fraud-proof, tamper-proof, 100% secure, guaranteed, official credits, "measured" (unless it comes from a stored benchmark or validation run), "live GPS" or "live traffic" (while data is simulated). Put **Estimated** next to every emission, saving or credit figure.

10. Business rules and validation

CO_2 allowance bands (per truck and fleet): used % = month-to-date kg / limit. 100% or more = **Exceeded** (red); 95% to below 100% = **Critical** (red); 80% to below 95% = **Warning** (amber); below 80% = **On track** (green). Show a toast when a band is crossed upwards (once per band per month) and create an `allowance` alert. Month-end forecast = (month-to-date ÷ days elapsed) × days in month.

Alert rules

Alert	Rule
Early warning	Section 6.4; 5-minute cooldown per truck
Idling	Idle at least N minutes (setting, default 10); once per episode
Offline	No heartbeat or record for 10 minutes while a trip is open
Integrity	When a truck's status changes to Check needed
High estimated CO_2 rate	Rolling 5-minute CO_2 rate above 1.5 times the truck's 7-day median rate (our definition; adjust after measuring)
Allowance	Band crossing (above)

Trip rules. A trip can end only when speed is 5 km/h or below ("Stop the truck to end the trip"). Eco-route suggestions follow the toggle on trip setup. A route event is recorded for both Take eco-route and Stay on current route (`accepted` true or false). Declared cargo

can change only at trip start or at a stop.

Form validation (server and client, same Zod schemas)

Form	Rules and messages
Login	Valid email ("Please enter a valid email."); wrong credentials show "Email or password is incorrect."; production passwords at least 8 characters
Driver login	Driver ID plus 4-digit PIN; "Driver ID or PIN is incorrect."; lockout after 5 failures
Add truck	Plate uppercase, matches `^[A-Z0-9][A-Z0-9 -]{3,11}$` ("Enter a valid plate (4-12 letters/digits).") and unique ("This plate is already registered."); Model 2 to 40 characters ("Model is required (2-40 characters)."); Driver optional, matches `^[A-Za-z][A-Za-z .'-]{1,39}$` ("Driver name is not valid."); Allowance above 0 and at most 100,000 kg ("CO2 allowance must be a positive number (max 100,000 kg)."); Starting odometer 0 or more ("Odometer cannot be negative.")
Edit allowance	"Enter a positive number (max 100,000)."
Fuel log	Litres 0.1 to 2,000 ("Fuel must be between 0.1 and 2,000 L (no negative values)."); Odometer 0 to 2,000,000 and not lower than the last reading ("Odometer cannot be lower than the last reading ({n} km)."); Date valid, not in the future (60 s tolerance), not before 2020-01-01; Cost optional, 0 or more
Vehicle profile	Name required; efficiency above 0 ("Enter a name and an efficiency above 0"); keep at least one profile
Multi-stop	At least 2 stops ("Keep at least 2 stops")

Offline rules. Fuel logs, route events, GPS points and telemetry always save locally first; the UI never blocks on the network; items created offline show "Pending sync" until acknowledged.

11. Security, privacy and honesty

- **Server-side authorisation on every route.** Check role and `company_id` in the handler or a shared guard; never trust IDs from the client. Test tenant isolation explicitly.
- **Secrets**: hashed passwords and PINs, device tokens stored as hashes, short expiry and revocation, no secrets in the client bundle or logs.
- **Abuse protection**: rate limits on login, PIN login and ingest; Zod validation with unknown fields rejected; body size limits; CSRF protection through SameSite cookies and Auth.js; security headers (CSP, HSTS, X-Content-Type-Options, Referrer-Policy, frame-ancestors none).
- **Privacy**: location is recorded only during an open trip and the driver app says so at first login; keep raw telemetry 14 days; support "export my data" and "delete my account" for drivers [ADDED]; store only what the screens need.
- **Audit log** for logins, setting changes, truck and driver changes, integrity verification, tamper demo and data exports.
- **Supply chain**: pin exact dependency versions, commit the lockfile, avoid brand-new releases.
- **Honesty rules (project-level)**: all emissions, savings and credits are estimates; credits are not official; the traffic, positions and OBD signals are simulated unless a real adapter is connected; targets are never shown as achieved results; integrity checks are tamper-evidence, not proof of sensor accuracy.

12. What was missing in the prototype and is now included

Real backend and database with migrations; authentication, roles and tenant isolation; password reset; driver management and PIN reset; ingest API with idempotency; shared emission, idle, risk and GA code; the telemetry simulator and a seed that uses the real pipeline; real SHA-256 hash chain, verification job and tamper demo; rollup tables and an alert engine; persisted settings and allowances (the prototype used `localStorage`); trip lifecycle and trip plans; audit log; Research page with edge vs cloud, outage and VED validation; fuel reconciliation; demo control panel; PWA install and service worker; real i18n with RTL; rate limiting and security headers; Support, Privacy and Terms pages; health endpoint; retention job; automated tests and CI; README and deployment runbook.

13. Testing and acceptance

Automated tests

- **Unit (Vitest)**: emission model with known inputs; idle detection edge cases (speed exactly 5 km/h); risk bands and sensitivity shifts; allowance bands at 79.9 / 80 / 94.9 / 95 / 99.9 / 100%; GA never returns a route worse than its seeds, respects the 2.0 s cap and is reproducible with a fixed seed; hash chain golden vectors (changing **any** field of any record breaks verification); **byte-for-byte equality** between the device hash and the server hash for the same record; outbox re-send is idempotent.
- **Property tests (fast-check)**: random record sequences with random duplicate and out-of-order delivery always end with the same stored chain and no double-counted rollups.
- **Integration**: ingest with duplicates, conflicts, partial failure and a broken `prevHash`; tenant isolation (a manager can never read another company); rate limits; cron routes reject calls without `CRON_SECRET`.
- **End-to-end (Playwright)**: (1) login → truck → Monthly Report → Day Detail drill-down; (2) driver starts a trip, congestion appears, warning shows, **Take eco-route**, and the event appears on the manager's Route Savings; (3) toggle network loss, create records and a fuel log, restore, and see everything arrive with the truck still Verified; (4) tamper with one stored record, press Verify integrity, see **Check needed** with the correct record number; (5) CSV export opens correctly; (6) Urdu RTL layout; (7) phone-width layout.

Acceptance criteria for the research claims. Do not pre-commit to numbers; the page must show whatever is measured and label it so.

Claim	How acceptance is shown
Offline recovery	10-minute simulated outage run at least 30 times: recovery rate and duplicate count stored in `benchmark_runs` and charted
GA speed on the edge	`elapsedMs` distribution on the real Android phone and on a laptop; share of runs within 2.0 s reported as measured
Tamper evidence	50 or more random single-field tamper attempts across random records; detection rate reported, with the exact failing record number
Emission model	VED validation run imported and displayed with MAE, RMSE, MAPE, R^2 and subset size

Claim	How acceptance is shown
Edge vs cloud	Latency, bandwidth, failure behaviour, route quality and resource use for both modes, with confidence intervals and the region and device used
Quality gates	No critical axe accessibility issues; Lighthouse PWA installable; no secrets in the repo; all pages show loading, empty and error states

14. Phased build plan

Phases follow the Web Features priority (truck-to-day path first, then login, Route Savings and Integrity, then the rest) but start with the plumbing that those screens need. The proposal's 6-month plan has room for all phases.

Phase	Build	Done when
0. Foundation	Next.js scaffold, tokens and AppShell, auth (manager login, roles), Postgres + Drizzle, CI, first Vercel preview deploy, `/api/v1/health`	You can sign in on the Vercel URL and see an empty dashboard shell
1. Data and manager core	Schema, `packages/core` emission + simulator, seed script using the real pipeline, pages W2 Dashboard, W3 Fleet, W4 Truck Detail, W5 Monthly Report, W6 Day Detail reading from the database	The whole truck-to-day drill-down shows seeded, calculated numbers (no hard-coded values)
2. Driver PWA and sync	Driver login, Home, Trip, Live trip, simulator, Dexie outbox, ingest endpoint, rollups, network badge, Sync page, Fuel Log	Records created in the driver app appear on the manager dashboard within one poll; offline records arrive after reconnecting

Phase	Build	Done when
3. Integrity and alerts	Hash chain on device and server, verification job and button, Integrity page, alert engine, Alerts page, drawer	Tampering a record flips the truck to Check needed at the right record
4. Risk, GA and eco-routes	Risk score, GA worker, eco-route decision screen, route events, Route Savings page, Multi-stop Planning, GA run storage	A simulated congestion zone triggers a warning, a GA result within the time cap, and a saving visible to the manager
5. Credits, reports, map, admin	Carbon Credits and allowances, Reports and exports (CSV/PDF), Live Map, Fuel & Fleet Logs with reconciliation, Vehicle Profiles, Drivers, Settings	Every menu item works with real data; exports download
6. Research page	Edge vs Cloud runner, outage test, tamper demo page, VED import, GA performance, Related work	Each tab shows stored, labelled results
7. Polish and launch	Urdu RTL, PWA install, accessibility pass, empty/error states, security headers, performance, README, demo script, production deploy and smoke test	All acceptance rows in Section 13 pass or are honestly reported

15. Deployment on Vercel

1. Push the repo to GitHub and **import it in Vercel** (framework preset Next.js).
2. In the project's Storage / Marketplace tab **add Neon Postgres** in the region closest to your users (the connection string is injected as `DATABASE_URL`). Use a separate database branch for preview deployments.
3. Add the environment variables from Section 7.4 for Production and Preview. Generate long random values for `AUTH_SECRET`, `DEVICE_TOKEN_SECRET` and `CRON_SECRET`. Set `DEMO_MODE=true` for the demo deployment.

4. **Migrations**: run `drizzle-kit migrate` as part of the build or as a manual release step. **Seed**: run the seed script locally against the target `DATABASE_URL`, or call the protected `/api/v1/admin/seed` once.
5. Add `vercel.json` (Hobby allows daily cron only; times are UTC and may fire anywhere inside the hour):

```
{
  "crons": [
    { "path": "/api/cron/verify-integrity", "schedule": "0 21 * * *" },
    { "path": "/api/cron/rollup-repair", "schedule": "30 21 * * *" }
  ]
}
```

Put retention cleanup inside `rollup-repair` so only two cron jobs are needed. Set `maxDuration` per route (Section 7.2) and the function region next to the database. 6. **Service worker rules**: never cache `/api/*`; serve `sw.js` with `Cache-Control: no-cache`; version the cache so updates do not strand drivers on old code. 7. Attach your domain (HTTPS is automatic). Open the Vercel URL on the Android phone and **Add to Home screen** to test install and offline. 8. **Smoke test after every deploy**: `/api/v1/health`, manager login, driver login, one ingest batch, one cron call with the secret, one CSV export. 9. Rollback with Vercel's instant rollback to a previous deployment; database migrations should be backward-compatible for one release. 10. Turn on Vercel Logs and Analytics; optional Sentry for error tracking.

16. Decisions the team should confirm

1. **Credit definition**: confirm the kg-based allowance and the tonne-based "Estimated credits" shown together (Section 2.3, item 1).
2. **Plan**: Hobby is non-commercial and has daily cron only. Is that acceptable for the defence, or will someone sponsor Pro?
3. **Urdu**: required for the demo, or English only first?
4. **Edge device for timing**: which Android phone will you measure on? The budget lists one at PKR 50,000.
5. **Traffic**: simulated only, or do you want a real traffic source later (clearly labelled)?
6. **Fleet size**: keep 8 trucks for the demo or seed more to show pagination?
7. **Citations**: confirm the DOI, venue and year of Cheng and Jia and of Bai et al. before submission (Section 2.3, item 9).

8. **Examiner access**: do you want a read-only examiner account on the live demo?

Appendix A. Seed data from prototype v5

Company: AJ Logistics. **Manager (demo only):** Aamir Joyo, `aamir.joyo@ecoroute.com`. **Driver (demo only):** Bilal Ahmed, Driver ID `PK-094`.

Truck	Model	Plate	Driver	Status	Distance today (km)	CO_2 today (kg)	Declared cargo (kg)	Integrity	Monthly allowance (kg)
TRUCK_PK_094	Hino 500	TK-5021	Bilal Ahmed	Active	468	412	3,500	Verified	6,000
TRUCK_PK_031	Isuzu F-Series	TK-3118	Imran Shah	Active	500	520	2,800	Verified	8,000
TRUCK_PK_108	Hino 300	TK-8108	Usman Ali	Active	460	480	3,000	**Check needed**	7,000
TRUCK_PK_052	Hyundai Mighty	TK-5204	Naveed Khan	Offline	0	0	1,800	Pending	3,000
TRUCK_PK_076	Isuzu F-Series	TK-7610	Shahid Hussain	Active	420	440	2,500	Verified	6,500
TRUCK_PK_021	Hino 500	TK-2109	Kashif Raza	Active	480	500	3,100	Verified	7,500
TRUCK_PK_064	Mitsubishi Fuso	TK-6401	Sajid Ali	Active	500	560	2,200	Verified	7,000
TRUCK_PK_011	Hino 300	TK-1102	Asif Memon	Idle	122	208	0	Verified	4,000

These are **targets for the simulator's output**, not numbers to hard-code: the seeded history should produce roughly these daily totals (fleet about 3,120 kg CO_2, 2,950 km). The prototype's October month-to-date for TRUCK_PK_094 (about 4,940 kg over 12 days against a 6,000 kg limit = Warning band) is a good test case.

Alerts to reproduce (type, text, truck, time): Early warning "Slow traffic likely ahead near the bypass" TRUCK_PK_094 10:42; Idling "Idling over 10 minutes" TRUCK_PK_011 09:15; Offline "Truck offline for 12 min" TRUCK_PK_052 08:50; Integrity "Integrity check needed" TRUCK_PK_108 08:15; Early warning "Early warning: bottleneck ahead" TRUCK_PK_031 07:40.

Eco-route example (TRUCK_PK_094, 10:42): current route 18.4 km, 31 min, 7.2 L, 19 kg CO_2; eco route 19.1 km, 20 min, 1.2 L, 3 kg CO_2; saving 6 L, 16 kg, 11 min. Your GA and fuel model should be able to **produce** a result of this shape from a congestion scenario; do not store

these as constants.

Demo route names for Top routes by savings: Karachi to Hyderabad (M-9), Hyderabad to Nawabshah (N-5), Nawabshah to Sukkur, Lahore Ring Road bypass, Hala Bypass eco-route. **Trip id examples:** `TRIP_KHI_HYD_102` (Karachi to Hyderabad), `TRIP_HYD_NWB_103`.

Vehicle profiles: Hino 500 (Diesel) Fuel 30 L/100 km; Isuzu Hybrid Hybrid 22 L/100 km; Eco EV Van EV 38 kWh/100 km. **Offline map packs:** Sindh corridor 182 MB, Punjab corridor 240 MB, Karachi city 64 MB.

Appendix B. Toasts and short messages

Eco-route taken: "Eco-route taken. Route event recorded." · Saved offline: "Saved offline. It will sync automatically." · Online save: "Fuel log saved" · Truck added: "Truck added: {plate}" · Allowance saved: "Allowance updated" · Settings: "Changes saved" · Stops: "Stops reordered (estimated lowest-emission order)" · Truck not stopped: "Stop the truck to end the trip" · Sync idle: "Everything is already synchronized" · Network lost: "You are offline. Data is saved on this device." · Sync success banner: "Network connection restored. Offlined trip logs and CO2 telemetry data have been successfully synchronized with the Fleet Manager server." · OBD connected: "OBD-II interface connected. Live ECU telemetry is now streaming." (change "Live" to "Simulated" while `src = sim`) · OBD lost: "OBD-II signal lost. Check the dongle or OBD port. Telemetry is paused."

Appendix C. Master prompt for the AI builder

> You are a senior full-stack engineer. Build **EcoRoute Fleet** exactly as described in the attached specification. Target: Next.js (App Router, TypeScript strict) deployed on Vercel, Postgres (Neon) with Drizzle, Auth.js, Tailwind + shadcn/ui, Recharts, react-leaflet, Dexie, Serwist, Vitest, Playwright.
>
> Work **only on Phase {N}** from Section 14. Before coding, read the whole spec and list any blocking ambiguities (maximum three); otherwise state your assumptions and proceed.
>
> Rules: (1) no hard-coded demo numbers in UI code; every figure comes from the database or a calculation over it; (2) put emission, idle, risk, GA, graph, simulator and hash-chain code in a shared pure-TypeScript module used by the device, the server and the seed script; (3) follow the UI wording rules in Section 9.6 and label all emission, saving and credit figures "Estimated"; (4) never claim tamper-proof, official credits, live GPS or live traffic; (5) do not add features that are not in the spec; mark anything you must assume; (6) pin exact dependency versions and commit the lockfile; (7) write tests for everything in Section 13 that belongs to this phase.
>
> When finished, report: files created or changed, how to run and test locally, how to deploy this phase to Vercel, and what is **not** done yet.